arXiv:2611.02613v1 [cs.RO] 2 Nov 2026

MAPO: Mixture of Experts with Alternating Policy Optimization for Universal Humanoid Motion Tracking

Seonghyeon Lim[1] **Hyeonwoo Lee**[1] **Mincheol Kim**[2] **Kimin Lee**[3,4†] **Hyun Myung**[1†]
[1]School of Electrical Engineering, KAIST [2]Robotics Program, KAIST
[3]Graduate School of AI, KAIST [4]Config
{shlim, hyeonwoolee, kmc96, kiminlee, hmyung}@kaist.ac.kr

Abstract: Universal humanoid motion tracking is essential for enabling humanoid robots to perform diverse tasks in human environments. To capture the wide diversity of human motions, mixture-of-experts (MoE) policies have been widely adopted, leveraging dynamic routing to enhance policy expressiveness. However, we identify that divergence between old and updated policies can be misestimated during policy-gradient updates of MoE policies, due to their dynamic routing. Because most policy-gradient algorithms regulate policy updates based on the divergence, this misestimation can lead to either overly conservative or excessively large updates, which can yield a suboptimal policy. In this paper, we propose Mixture of Experts with Alternating Policy Optimization (MAPO), which alternately optimizes the router and the experts to avoid the divergence misestimation. MAPO consistently tracks 182 motions that PPO with MoE fails to track, whereas PPO with MoE tracks only one motion that MAPO fails to track. MAPO also attains the highest tracking success rate among the compared methods and the lowest tracking error across most error metrics, demonstrating its superior performance. Videos can be found on the project website: https://mapo-corl2026.github.io/MAPO/

Keywords: Humanoid Control, Mixture of Experts, Policy Gradient



Figure 1: MAPO resolves the divergence misestimation problem that hinders policy-gradient training of MoE policies, enabling effective humanoid motion tracking. Consequently, MAPO successfully tracks a wide variety of human motions, including complex whole-body motions, such as (a) sitting in mid-air, (b) abrupt full-body turns, (c) deep crouching, and (d) one-footed spinning. Red spheres denote the reference motion targets.

†Corresponding authors.

1 Introduction

Humanoid robots have the potential to perform various tasks in human environments, given their human-like morphology and whole-body dexterity for locomotion and manipulation. To achieve such potential, a humanoid should be able to perform diverse motions ranging from basic locomotion to complex whole-body behaviors. Reinforcement learning (RL)-based approaches have enabled humanoids to perform robust walking [1, 2, 3, 4], parkour [5, 6], and fall recovery [7, 8]. However, extending such approaches to universal humanoid motion tracking requires physically feasible and diverse reference motions during training. Large-scale human motion datasets [9, 10, 11] have become promising sources of such reference motions [12, 13, 14, 15, 16, 17, 18].

Nevertheless, learning a universal motion tracker with a monolithic policy can be challenging [19, 20]. Because diverse reference motions require distinct control strategies, this task can be cast as multi-task learning, where monolithic policies suffer from inter-task interference [21, 22]. Moreover, different motion modes often share overlapping state regions [19], likely exacerbating the interference [23]. In contrast, the mixture of experts (MoE) architecture [24] has multiple experts and a router as its components, with the router computing mixing weights over the experts for a given state. This architecture has shown effectiveness in such multi-task learning problems [25, 26, 27, 28, 29]. The MoE architecture has also been explored for motion tracking tasks [19, 20, 30, 31]. However, these studies primarily focus on architectural design, and the problems that arise when MoE policies are trained with widely used policy-gradient algorithms [32, 33] remain largely unexplored.

In this work, we identify a phenomenon termed *divergence misestimation* where the divergence between the data-collection and updated policies does not accurately reflect the magnitude of MoE policy updates. Specifically, the phenomenon becomes problematic for algorithms that constrain policy updates via this divergence, including proximal policy optimization (PPO) [33], which is widely adopted in robot control tasks. For monolithic policies, the magnitude of the policy update is directly reflected in the divergence. However, the distribution of an MoE policy is a weighted sum of expert outputs, with mixing weights determined by the router. Therefore, the divergence can change not only with updates to the experts but also with shifts in the mixing weights. It can thus either overestimate or underestimate the changes in each component, leading to overly conservative or aggressive policy updates, respectively.

To address the divergence misestimation, we propose Mixture of Experts with Alternating Policy Optimization (MAPO). A naive approach to resolve the divergence misestimation is to freeze the mixing weights at their data-collection values during the update. Although this approach can avoid the misestimation, it prevents the router from being optimized via the RL objective. MAPO resolves this trade-off by symmetrically alternating updates of the two components. During the expert update, the mixing weights are held fixed at their rollout values, and vice versa for the router update. In this way, MAPO measures the divergence only for the component under update, rather than over the entire MoE policy as in existing policy-gradient algorithms. Such separation avoids the misestimation, and the alternating updates allow both components to be optimized with the RL objective.

Our contributions are summarized as follows. (i) We identify divergence misestimation in MoE policy optimization. Because most policy-gradient algorithms regulate policy divergence, the misestimation can lead to overly conservative or aggressive updates. (ii) We propose MAPO, a simple but efficient remedy for the divergence misestimation that alternates expert and router updates, measuring divergence per component. In our experiments on humanoid motion tracking, the proposed method achieves substantially higher reward and better tracking performance than a PPO-trained MoE policy by avoiding the divergence misestimation (see Figure 1).

2 Related Work

Mixture of Experts in Reinforcement Learning: Originally proposed by Jacobs et al. [24], MoE has gained increasing attention for scaling up large language models [34, 35, 36, 37] and

vision models [38]. MoE has also been explored in RL [39, 40, 41], particularly in multi-task settings [25, 26, 27, 28, 29]. Many of these works adopt a sparse MoE, which activates only a subset of experts for each input, enabling the network to scale efficiently. However, sparse routing is non-differentiable, hindering end-to-end optimization [39]. In contrast, dense MoE [42, 43, 44, 45] uses a weighted combination of all experts with continuous mixing weights, making it fully differentiable [42]. Recent empirical studies [25, 46] have analyzed MoE in RL, reporting that differentiable MoE shows better performance than sparse MoE. Despite these advances, the divergence misestimation phenomenon has remained unaddressed, which we identify and resolve in this work.

Universal Whole-body Control for Humanoids: RL-based approaches have achieved robust whole-body control for quadrupeds [47, 48] and legged manipulators [49, 50, 51, 52, 53]. Such approaches require sampling motion targets during training, which is straightforward when motion targets can be abstracted as low-dimensional states, such as body velocities or end-effector poses. However, this approach is hard to extend to humanoids, where such sampling rarely yields physically feasible whole-body motions. Instead, DeepMimic [12] demonstrates that human motion data can serve as feasible and diverse reference motions for various simulated avatars. This paradigm has been scaled to achieve universal humanoid whole-body control in the real world [12, 13, 14, 15, 16, 17, 18, 54, 55, 56, 57, 58, 59]. However, the increasing scale and diversity of motion data raise the need for more expressive policy networks. MoE architectures have been explored to improve expressiveness via expert specialization [19, 20, 30, 31]. CLONE [31] and KungfuBot2 [30] adopt a sparse MoE, whose non-differentiable routing hinders end-to-end RL optimization. In contrast, GMT [19] and FAST [20] employ a fully differentiable dense MoE. However, these works primarily focus on architectural design, leaving the effective optimization of MoE policies under widely used policy-gradient algorithms [32, 33] underexplored.

3 Background: Mixture of Experts with Policy Gradient

3.1 Reinforcement Learning

RL considers a sequential decision-making problem within a given environment [60]. The environment is typically modeled as a Markov decision process defined by the tuple $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s,a)$ is the transition probability from state s to s' given action a, $R(s,a)$ is the reward function, and $\gamma \in [0,1]$ is the discount factor. A stochastic policy $\pi(a|s)$ defines a distribution over actions conditioned on the current state. Because the desired behavior is encoded in the reward function, the goal of RL is to find an optimal policy π^* that maximizes the expected return, $\mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s,a)\right]$.

3.2 Policy Gradient Methods

Policy gradient methods directly optimize the policy to maximize the expected return. One of the key challenges in policy gradient methods is that aggressive updates can destabilize training. To address this challenge, Schulman et al. [32] proposed trust region policy optimization (TRPO), which constrains the Kullback–Leibler (KL) divergence between the data-collection policy π^{old} and the new policy π^{new}. When the divergence remains under the trust region, TRPO theoretically guarantees monotonic improvement of the policy performance. However, TRPO is computationally expensive because it requires second-order optimization. To address this limitation, PPO [33] was proposed as an efficient first-order alternative, using the following clipped surrogate objective L^{PPO}:

$$L^{\text{PPO}} = \hat{\mathbb{E}}_t \left[\min \left(r_t \, \hat{A}_t, \, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) \, \hat{A}_t \right) \right], \tag{1}$$

where $r_t = \pi^{\text{new}}(a_t|s_t)/\pi^{\text{old}}(a_t|s_t)$ is the probability ratio between the new and old policies, $\hat{A}_t$ is an advantage estimate, and ϵ controls the clipping range. Notably, the PPO clipping induces an implicit trust region by constraining the KL divergence on the order of ϵ^2 (see Supplementary A).

4 Divergence Misestimation in Mixture-of-Experts Policy Optimization

The divergence of MoE policies can be misestimated. Such misestimation is problematic for policy-gradient algorithms [32, 33] that regulate the divergence. To formalize the misestimation, we consider a Gaussian policy whose mean is parameterized by a dense MoE. Let $\mu_1, \ldots, \mu_N$ denote the outputs of N experts. The MoE policy is then parameterized as $\pi(a|s) = \mathcal{N}\big(m,\, \sigma^2 I\big) = \mathcal{N}\Big(\sum_{e=1}^{N} w_e\, \mu_e,\; \sigma^2 I\Big)$, where $\mathbf{w} = [w_1, \ldots, w_N]$ are obtained by applying a softmax to the router's output, ensuring their sum to one. For simplicity, σ is assumed to be fixed within each update step.

The measured KL divergence between the old and new policies is as follows:

$$\mathrm{KL}_{\text{measured}} = D(\pi^{\text{old}} \,\|\, \pi^{\text{new}}) = \|m^{\text{old}} - m^{\text{new}}\|^2 / 2\sigma^2, \tag{2}$$

where $m^{\text{old}} = \sum w_e^{\text{old}} \mu_e^{\text{old}}$ and $m^{\text{new}} = \sum w_e^{\text{new}} \mu_e^{\text{new}}$ denote the means of the old and new policies, respectively. The difference $m^{\text{old}} - m^{\text{new}}$ yields the decomposition as follows:

$$m^{\text{old}} - m^{\text{new}} = \underbrace{\sum w_e^{\text{old}} \left(\mu_e^{\text{old}} - \mu_e^{\text{new}}\right)}_{d_\mu} + \underbrace{\sum \left(w_e^{\text{old}} - w_e^{\text{new}}\right) \mu_e^{\text{new}}}_{d_w}, \tag{3}$$

where d_μ and d_w isolate the changes attributable to the experts and the router, respectively. Then, the measured KL divergence can be expressed as follows:

$$\mathrm{KL}_{\text{measured}} = \frac{1}{2\sigma^2} \left(\|d_\mu\|^2 + 2\, d_\mu \cdot d_w + \|d_w\|^2 \right). \tag{4}$$

We now show that the measured KL divergence can misestimate the true changes in each component. To measure the true changes in each component, we decompose the policy update into two sequential steps that update the experts first and then the router. Consider a partially updated policy, $\pi' = \mathcal{N}\left(\sum w_e^{\text{old}} \mu_e^{\text{new}}, \sigma^2 I\right)$, in which only the experts are updated while the router remains unchanged. The fully updated policy π^{new} is then obtained by updating only the router from π'. The two-step path is as follows: $\pi^{\text{old}} \xrightarrow{\text{expert update}} \pi' \xrightarrow{\text{router update}} \pi^{\text{new}}$.

Then we can define the KL divergence for each update step in the path as follows:

$$\mathrm{KL}_{\text{expert}} = D(\pi^{\text{old}} \,\|\, \pi') = \frac{\|d_\mu\|^2}{2\sigma^2}, \qquad \mathrm{KL}_{\text{router}} = D(\pi' \,\|\, \pi^{\text{new}}) = \frac{\|d_w\|^2}{2\sigma^2}. \tag{5}$$

Therefore, the measured KL divergence can be rewritten as follows:

$$\mathrm{KL}_{\text{measured}} = \mathrm{KL}_{\text{router}} + \mathrm{KL}_{\text{expert}} + d_w \cdot d_\mu / \sigma^2. \tag{6}$$

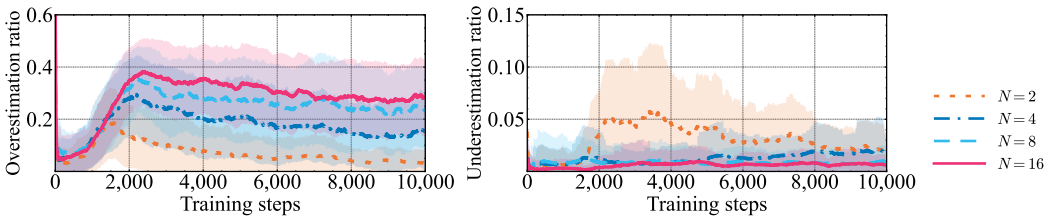
An analogous decomposition holds when the router is updated first, with d_w and d_μ defined along that path.

Definition 1 (Divergence Misestimation)**.** *Divergence misestimation* refers to the discrepancy between the measured divergence $\mathrm{KL}_{\text{measured}}$ and the sum of component-wise divergences $\mathrm{KL}_{\text{router}} + \mathrm{KL}_{\text{expert}}$. Specifically,

$$\begin{aligned} d_w \cdot d_\mu > 0 \;&\Longrightarrow\; \mathrm{KL}_{\text{measured}} > \mathrm{KL}_{\text{router}} + \mathrm{KL}_{\text{expert}} \quad \text{(overestimation)}, \\ d_w \cdot d_\mu < 0 \;&\Longrightarrow\; \mathrm{KL}_{\text{measured}} < \mathrm{KL}_{\text{router}} + \mathrm{KL}_{\text{expert}} \quad \text{(underestimation)}. \end{aligned}$$

When overestimation occurs, the policy update is overly constrained by the divergence regularization, which can lead to conservative updates and suboptimal performance. Underestimation has the opposite effect. It makes the divergence regularization fail to constrain aggressive updates of each component, potentially leading to unstable training. Notably, the sign of $d_w \cdot d_\mu$ is difficult to control explicitly during policy updates, as the gradient directions of the router and experts are not guaranteed to align or oppose each other. Therefore, simple solutions such as adjusting the clipping range of PPO cannot effectively address the divergence misestimation.

Figure 2 shows the ratio of non-trivial divergence misestimation during the training of an MoE policy with vanilla policy-gradient algorithms. A divergence misestimation is considered non-trivial when



(a) Overestimation (b) Underestimation

Figure 2: Ratio of non-trivial policy divergence misestimation. N is the number of experts. Overestimation becomes more frequent with more experts, whereas underestimation becomes more prevalent with fewer experts. Triviality threshold is ϵ^2, where ϵ is the default PPO clipping ratio.

the difference between the measured KL divergence and the sum of component KL divergences exceeds the threshold. Because PPO typically constrains the KL divergence on the order of ϵ^2, we set the threshold to ϵ^2 with the default value of ϵ. Overestimation becomes more prevalent as the number of experts increases, while underestimation dominates with fewer experts. Therefore, divergence misestimation is inherent to combining MoE with policy-gradient optimization; tuning the number of experts only switches its dominant mode between underestimation and overestimation. Please refer to the Figure D.2 for the distribution of the misestimation and further analysis of its effect under training dynamics.

5 Mixture of Experts for Universal Humanoid Motion Tracking

In this study, we aim to train a humanoid robot to imitate diverse human motions [9, 10, 11]. Following previous works [19, 20, 30, 31], we formulate this task as an RL problem and adopt an MoE architecture to enhance the expressiveness of the policy network. To avoid the divergence misestimation issue, we propose MAPO, a policy optimization algorithm for MoE policies.

5.1 MAPO: Mixture of Experts with Alternating Policy Optimization

A simple solution to policy divergence misestimation is to store the mixing weights during data collection and use them to compute the probability ratio during policy optimization. However, in this case, the router is no longer part of the gradient computation and thus cannot be optimized with the RL objective. We thus face a trade-off between preventing divergence misestimation and optimizing the router with the RL objective.

To address this trade-off, we propose MAPO. A key idea of MAPO is to alternate between expert and router updates, with the other component held fixed at its data-collection value. The corresponding probability ratios for each update are

$$r_t^{\text{expert}} = \frac{\mathcal{N}(\sum w_e^{\text{old}} \mu_e, \sigma^2 I)}{\mathcal{N}(\sum w_e^{\text{old}} \mu_e^{\text{old}}, \sigma^2 I)}, \quad r_t^{\text{router}} = \frac{\mathcal{N}(\sum w_e \mu_e^{\text{old}}, \sigma^2 I)}{\mathcal{N}(\sum w_e^{\text{old}} \mu_e^{\text{old}}, \sigma^2 I)}. \tag{7}$$

Following Equation 1, the surrogate loss is then computed separately for each component using the clipped probability ratio, yielding gradients for the corresponding parameters. Because each ratio can directly measure the amount of change in the corresponding component, the policy divergence misestimation can be prevented. Additionally, because both components are used to compute the gradient, the router and experts can be optimized with the RL objective. Notably, *alternating* refers to separate gradient computations for the experts and the router, rather than an interleaved update schedule. Both surrogate losses are computed from the same rollout data within a single update step.

Figure 3 illustrates the differences in update mechanisms between MAPO and a standard policy gradient algorithm, PPO. Specifically, PPO measures the KL divergence of the mixed policy, while MAPO measures each component separately for clipping. When overestimation occurs, the measured divergence of the mixed policy can exceed the implicit trust region even though each component's update is admissible. In this case, MAPO still updates both components properly, as shown in Figure 3 (a), whereas PPO blocks these admissible updates, as shown in Figure 3 (b). When underestimation occurs, the mixed policy can stay within the trust region while individual components

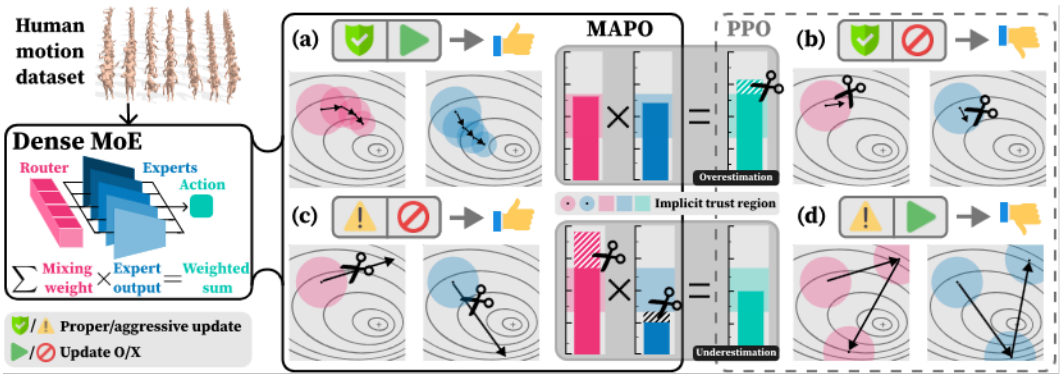


Figure 3: Comparison of update mechanisms between MAPO and PPO. Pink, blue, and blue-green indicate the router, experts, and mixed policy, respectively. The bars in the center show the magnitudes of each component and the mixed policy. The shaded regions show the implicit trust regions induced by clipping. (a, c) MAPO measures the KL divergence of each component separately, allowing proper regulation of the updates. (b, d) In contrast, because PPO measures the KL divergence of the mixed policy, the clipping mechanism may fail to regulate the updates properly.

violate their own trust regions. MAPO properly clips each component, as shown in Figure 3 (c), whereas PPO fails to clip the aggressive updates, as shown in Figure 3 (d). Consequently, MAPO avoids the divergence misestimation in both cases, enabling more effective policy optimization.

Although MAPO avoids the misestimation issue, each update step optimizes a partially updated policy rather than the fully updated policy, introducing a gradient bias. For the expert parameters, the bias preserves the stationary points of the true gradient. For the router parameters, the gradient direction may deviate, but the bias depends only on the relative inter-expert deviations and vanishes when experts update coherently. While the bias persists when experts update incoherently, our experiments show that resolving the misestimation yields a favorable net effect. See Supplementary B for a detailed analysis.

5.2 Teacher-Student Training Framework

We adopt the teacher-student training framework [14, 55, 56, 57], in which a teacher is trained with privileged information and then distilled into a student for real-world deployment.

Teacher Policy Training: The teacher policy is implemented as an MoE and trained with MAPO. For a fair comparison of the effect of MAPO, we closely follow prior humanoid motion tracking works [15, 17, 56, 58] for the observation space, reward function, and other training details [12]. The policy outputs target joint positions, which a PD controller converts into joint torques. The observation space consists of two parts. The first part, the proprioception state, contains the joint angles and velocities, root angular velocity, gravity vector, previous action, and per-link position, orientation, linear velocity, and angular velocity. The second part, the reference motion state, contains (i) future reference link positions and heading direction, (ii) current linear and angular velocities of each link, and (iii) per-link differences in current position and orientation between the robot and the reference. The reward function consists of tracking and regularization terms. The tracking term encourages the robot to imitate the reference motion, while the regularization term penalizes undesirable behaviors such as joint position limit violations. See Table C.1 for the full reward specification.

Student Policy Distillation: Because some privileged observations, such as the linear velocity of the robot, are unavailable in the real world [55], the teacher policy cannot be directly deployed. Therefore, we distill it into a student policy that excludes privileged observations. We use DAgger [61] for the distillation, where the student policy rolls out in the environment, and the teacher policy provides action labels for supervision. Because the student is trained with simple supervised learning, we adopt a monolithic network for it. The student policy's observation and domain randomization follow the settings from TWIST [16].

Table 1: Tracking metrics across methods. $E_{\text{g-pos}}$ and $E_{\text{l-pos}}$ are the global and root-relative (local) body position errors (mm), E_{acc} and E_{vel} are the body acceleration (mm/frame^2) and velocity (mm/frame) errors. $E_{\text{rt-h}}$, $E_{\text{rt-vel}}$, $E_{\text{rt-r}}$, and $E_{\text{rt-p}}$ denote the root height (m), linear velocity (m/s), and roll and pitch errors (rad), respectively. N denotes the number of experts. The best performance for each metric is highlighted in **bold**, and the second best is underlined.

	Method	Success ↑	$E_{\text{g-pos}}$ ↓	$E_{\text{l-pos}}$ ↓	E_{acc} ↓	E_{vel} ↓	$E_{\text{rt-h}}$ ↓	$E_{\text{rt-vel}}$ ↓	$E_{\text{rt-r}}$ ↓	$E_{\text{rt-p}}$ ↓
(a) Training set (AMASS [9] + LAFAN1 [10])										
	Monolithic PPO	81.78 %	165.7	96.50	5.787	16.35	0.060	0.532	0.169	0.136
	PPO with MoE	83.38 %	153.1	87.98	5.499	14.30	0.058	0.501	**0.120**	**0.105**
	MAPO with fixed router	83.78 %	152.7	86.65	5.250	14.42	0.052	0.487	0.144	0.122
	MAPO (Ours)	**86.60 %**	**147.4**	**78.10**	**4.576**	**11.78**	**0.048**	**0.406**	0.138	0.117
Ablation on N	$N = 4$ PPO with MoE	83.96 %	156.5	85.01	5.301	13.75	0.054	0.490	0.131	0.119
	$N = 4$ **MAPO**	**86.87 %**	**143.6**	**74.61**	**4.672**	**11.60**	**0.050**	**0.407**	**0.119**	**0.108**
	$N = 8$ PPO with MoE	82.28 %	164.4	93.47	5.841	15.51	0.064	0.538	**0.126**	**0.110**
	$N = 8$ **MAPO**	**87.85 %**	**134.8**	**71.65**	**4.120**	**10.06**	**0.045**	**0.360**	0.138	0.115
	$N = 16$ PPO with MoE	82.95 %	159.6	94.08	6.175	15.54	0.061	0.528	**0.118**	**0.107**
	$N = 16$ **MAPO**	**87.73 %**	**136.9**	**72.58**	**4.549**	**11.02**	**0.048**	**0.375**	0.128	0.110
(b) Test set (OMOMO [11])										
	Monolithic PPO	84.36 %	227.2	86.43	6.014	12.44	0.035	0.356	0.157	0.134
	PPO with MoE	82.78 %	241.4	93.90	6.451	13.77	0.041	0.397	0.140	0.130
	MAPO with fixed router	85.21 %	**221.6**	87.01	5.937	12.30	0.031	0.331	0.153	0.136
	MAPO (Ours)	**85.45 %**	225.8	**85.29**	5.764	**11.76**	**0.030**	**0.326**	0.151	0.129
	HOVER [56]	83.39 %	272.2	122.81	6.001	12.46	0.058	0.382	0.193	0.163
	TWIST 2 [58]	57.85 %	386.6	104.72	**4.661**	11.87	0.042	0.355	**0.080**	**0.074**

6 Experiments

6.1 Experiment Setup

We evaluate MAPO on a humanoid motion tracking task. Because MAPO focuses on the training of MoE teacher policies, all evaluations in this section are conducted on the teacher policy, except for the real-world deployment. We train both the teacher and student policies in the Isaac Lab simulator [62], with 4096 parallel environments on a single NVIDIA A6000 GPU. The student policy is deployed on a real humanoid robot, Unitree G1 (29DoF) [63], for real-world demonstration. We use AMASS [9] and LAFAN1 [10] datasets for training and OMOMO [11] for evaluation. All motions are retargeted to the humanoid using GMR [59] to account for morphological differences. The MoE policy uses 6 experts, following prior works [19, 30]. For all methods, we use the standard PPO hyperparameters; the clipping parameter ϵ is set to 0.2, and the learning rate is adaptively adjusted following Rudin et al. [64]. The other hyperparameters can be found in Table C.2.

We compare MAPO against the following baselines. **PPO with MoE**: A dense MoE policy trained with PPO. It directly captures how divergence misestimation affects MoE policy training. **Monolithic PPO**: A single monolithic policy trained with PPO. It serves as a baseline to quantify the benefit of the MoE architecture. **MAPO with fixed router**: A variant of MAPO that skips the router update, keeping the parameters of the router fixed. Although divergence misestimation is reduced by fixing the router, end-to-end router optimization is prevented. **HOVER** [56]: A universal motion tracker based on a teacher-student distillation framework. We re-implement their teacher policy on our humanoid platform and train it on the same dataset as our method. Notably, we scale the penalty terms of HOVER to 20% of the original values. The penalty adjustment is the minimum required to enable training without unrealistic motion. **TWIST 2** [58]: A state-of-the-art humanoid motion tracking method. We evaluate it using its released checkpoint.

6.2 Experimental Results

Tracking Performance: To evaluate the tracking performance, we measure the success rate and tracking errors in diverse metrics across all motions in the training and evaluation datasets, without any motion curation. A motion is considered successfully tracked if the average position tracking

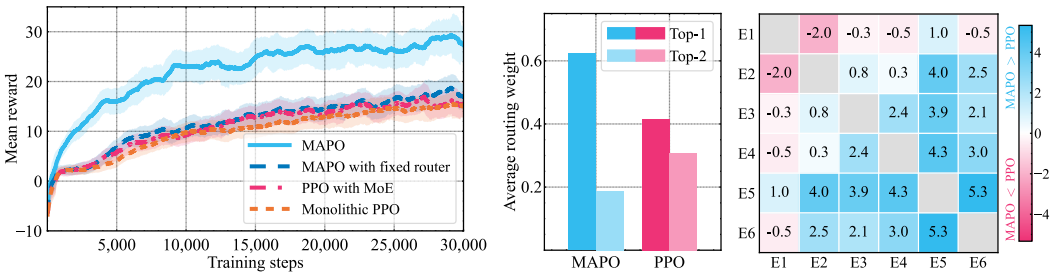
error is lower than 0.5 m throughout the motion, following previous works [54, 55]. For fairness, we freeze the tracking error at its failure-time value after a failure, because excluding post-failure errors would underestimate the error of harder motions that fail early. The results are summarized in Table 1. **MAPO** achieves the highest success rate and the lowest tracking errors across most metrics on both the training and evaluation datasets. Additionally, we ablate the number of experts in the MoE policy. As shown in Table 1, **MAPO** consistently achieves better performance than **PPO with MoE** across all tested expert counts, except for the root roll and pitch errors.

Effectiveness of MAPO Training: Figure 4 (a) shows the effect of divergence misestimation on training performance. **MAPO** achieves a significantly higher reward than **PPO with MoE**, demonstrating that avoiding misestimation substantially improves the training performance of MoE policies. Furthermore, **MAPO with fixed router** achieves slightly higher reward than **PPO with MoE**, suggesting that the performance loss from misestimation is comparable to that from fixing the router. Additionally, all MoE-based methods achieve higher reward than **Monolithic PPO**, confirming the effectiveness of the MoE architecture in humanoid motion tracking.

Expert Specialization: The specialization of dense MoE arises from differences in the mixing weights, which induce distinct gradients for each expert [24]. Therefore, sharper mixing weights are generally preferred, provided that all experts remain utilized across inputs. However, although sharpened routing is desirable for specialization, it can exacerbate divergence overestimation: sharpening induces a large router-side change d_w, which can amplify the cross term $d_w \cdot d_\mu$. Consequently, **PPO with MoE** tends to clip such updates, yielding less sharp routing than **MAPO** (Figure 4 (b)). The sharper routing of **MAPO** promotes better expert specialization, reflected in the generally larger pairwise expert output distances (Figure 4 (c)). Each entry is the average L2 distance between the outputs of expert pairs on the same input, reported as (**MAPO** $-$ **PPO**). Figure D.1 in the Supplementary shows per-expert activation across different motion types.

Ablation of Confounding Factors: We examine potential confounding factors through the ablation studies in Figure 5. First, we test whether the performance gain of **MAPO** results from a larger update budget due to separate clipping of the expert and router updates (Figure 5 (a)). We scale the default clipping parameter ϵ by 0.5 for **MAPO** and by 1.5 and 2 for **PPO with MoE**. Increasing ϵ reduces the performance of **PPO with MoE**. In contrast, **MAPO** with the smaller ϵ retains higher performance than **PPO with MoE** across all tested values. The results suggest that the performance gain of **MAPO** does not arise solely from a larger update budget.

Second, the policy standard deviation σ in **MAPO** is updated by both surrogate losses computed from the ratios in Equation 7. However, the entropy bonus contributes to the update of σ only once. Therefore, the effective entropy coefficient for σ can be halved. To separate the effect of the reduced effective entropy coefficient from the effect of reduced divergence misestimation, we introduce **Averaged-MAPO**. **Averaged-MAPO** scales each surrogate loss by $1/2$, thereby keeping the ef-



(a) (b) (c)

Figure 4: (a) Reward learning curves. MAPO achieves substantially higher rewards than the baselines. (b) Sorted mixing weights of MAPO and PPO, averaged at each rank across inputs. The misestimation of PPO prevents the mixing weights from sharpening. (c) Difference in pairwise expert output distances (MAPO $-$ PPO). Positive entries indicate higher expert diversity in MAPO.

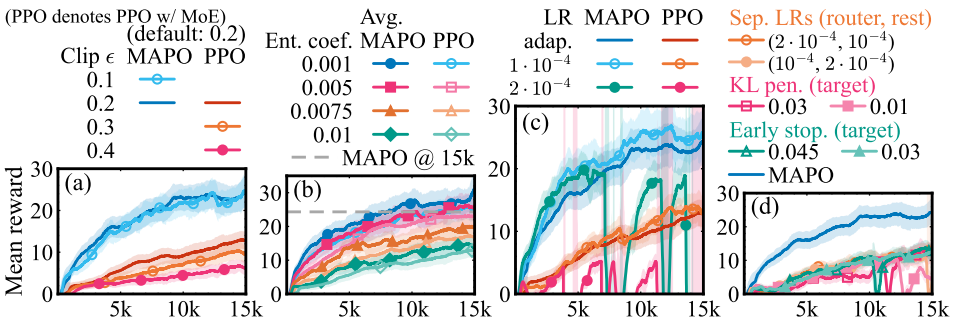


Figure 5: (a) Clipping parameter ϵ sweep. (b) Entropy coefficient sweep. (c) Fixed learning rate ablation. (d) Variants of PPO with MoE using a default learning rate of 10^{-4}.

fective entropy coefficient for σ unchanged. Under matched entropy coefficients, **Averaged-MAPO** consistently achieves higher reward than **PPO with MoE** across all tested values (Figure 5 (b)).

Consistency of Performance Gains: We examine whether the performance gain of **MAPO** persists under fixed learning rates. We compare **MAPO** and **PPO with MoE** at learning rates of 10^{-4} and 2×10^{-4} (Figure 5 (c)). **MAPO** consistently outperforms **PPO with MoE** at both learning rates. Because training becomes unstable for both methods at 2×10^{-4}, we set 2×10^{-4} as the maximum learning rate in the comparison. Additionally, we compare **MAPO** with variants of **PPO with MoE** (Figure 5 (d)). We test separate learning rates for the router and remaining parameters. We also evaluate alternative regularization techniques, including a KL penalty and early stopping. However, the tested variants do not substantially improve on the default **PPO with MoE** and fail to match the performance of **MAPO**.

Qualitative Results: Figure 1 visualizes representative motions where **MAPO** succeeds but **PPO with MoE** fails, under the same training setup. The selected motions require complex whole-body coordination, including sitting in mid-air, abrupt full-body turns, deep crouching, and one-footed spinning. These motions are selected from 182 cases on which **MAPO** consistently succeeds while **PPO with MoE** consistently fails across 5 independent trials, whereas the reverse occurs for only 1 motion. Notably, the five independent trials refer to evaluation rollouts of each trained policy. Additionally, we demonstrate that the **MAPO**-trained policy can serve as an effective teacher for distillation. Figure 1 also shows a real-world deployment of the student policy. The robot successfully tracks a wide range of reference motions, including challenging ones that require precise balance, such as squatting and bowing. Videos can be found on our project website.

7 Conclusion

In this study, we identified the divergence misestimation phenomenon that arises when MoE policies are trained with policy-gradient algorithms. To avoid the misestimation, we proposed MAPO, which alternately optimizes the router and experts, enabling more effective training and better tracking performance for humanoid whole-body control. The resulting policy performs complex whole-body motions well and is distilled into a deployable student policy on a real humanoid robot.

8 Limitations

While MAPO enables effective training of MoE policies with policy-gradient algorithms, further encouraging expert specialization remains an open question. One potential direction is to introduce an auxiliary loss that promotes discriminability between expert actions, inspired by prior works on unsupervised skill discovery [65, 66, 67, 68, 69]. Another limitation is that our analysis assumes dense softmax routing with σ fixed within each update step. The analysis does not cover learned per-expert variances or top-k sparse routing. Nevertheless, divergence misestimation may also occur in these settings.

References

[1] I. Radosavovic, T. Xiao, B. Zhang, T. Darrell, J. Malik, and K. Sreenath. Real-world humanoid locomotion with reinforcement learning. *Science Robotics*, 9(89):eadi9579, 2024.

[2] J. Long, J. Ren, M. Shi, Z. Wang, T. Huang, P. Luo, and J. Pang. Learning humanoid locomotion with perceptive internal model. In *Proc. IEEE International Conference on Robotics and Automation*, 2025.

[3] X. Gu, Y.-J. Wang, X. Zhu, C. Shi, Y. Guo, Y. Liu, and J. Chen. Advancing humanoid locomotion: Mastering challenging terrains with denoising world model learning. *arXiv preprint arXiv:2408.14472*, 2024.

[4] I. Radosavovic, S. Kamat, T. Darrell, and J. Malik. Learning humanoid locomotion over challenging terrain. *arXiv preprint arXiv:2410.03654*, 2024.

[5] Z. Zhuang, S. Yao, and H. Zhao. Humanoid parkour learning. In *Proc. Conference on Robot Learning*, 2025.

[6] S. Zhu, Z. Zhuang, M. Zhao, K.-Y. Lee, and H. Zhao. Hiking in the wild: A scalable perceptive parkour framework for humanoids. *arXiv preprint arXiv:2601.07718*, 2026.

[7] T. Huang, J. Ren, H. Wang, Z. Wang, Q. Ben, M. Wen, X. Chen, J. Li, and J. Pang. Learning humanoid standing-up control across diverse postures. *arXiv preprint arXiv:2502.08378*, 2025.

[8] X. He, R. Dong, Z. Chen, and S. Gupta. Learning getting-up policies for real-world humanoid robots. *arXiv preprint arXiv:2502.12152*, 2025.

[9] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black. AMASS: Archive of motion capture as surface shapes. In *Proc. IEEE/CVF International Conference on Computer Vision*, 2019.

[10] F. G. Harvey, M. Yurick, D. Nowrouzezahrai, and C. Pal. Robust motion in-betweening. *ACM Transactions on Graphics*, 39(4):60:1–60:12, 2020.

[11] J. Li, J. Wu, and C. K. Liu. Object motion guided human motion synthesis. *ACM Transactions on Graphics*, 42(6):1–11, 2023.

[12] X. B. Peng, P. Abbeel, S. Levine, and M. Van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions on Graphics*, 37 (4):1–14, 2018.

[13] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang. Expressive whole-body control for humanoid robots. *arXiv preprint arXiv:2402.16796*, 2024.

[14] M. Ji, X. Peng, F. Liu, J. Li, G. Yang, X. Cheng, and X. Wang. ExBody2: Advanced expressive humanoid whole-body control. In *Proc. Robotics: Science and Systems Workshop on Whole-body Control and Bimanual Manipulation: Applications in Humanoids and Beyond*, 2025.

[15] Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu. Beyond-Mimic: From motion tracking to versatile humanoid control via guided diffusion. *arXiv preprint arXiv:2508.08241*, 2025.

[16] Y. Ze, Z. Chen, J. P. Araujo, Z.-a. Cao, X. B. Peng, J. Wu, and K. Liu. TWIST: Teleoperated whole-body imitation system. In *Proc. Conference on Robot Learning*, 2025.

[17] Z. Luo, Y. Yuan, T. Wang, C. Li, S. Chen, F. Castaneda, Z.-A. Cao, J. Li, D. Minor, Q. Ben, et al. SONIC: Supersizing motion tracking for natural humanoid whole-body control. *arXiv preprint arXiv:2511.07820*, 2025.

[18] T. He, Z. Luo, W. Xiao, C. Zhang, K. Kitani, C. Liu, and G. Shi. Learning human-to-humanoid real-time whole-body teleoperation. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024.

[19] Z. Chen, M. Ji, X. Cheng, X. Peng, X. B. Peng, and X. Wang. GMT: General motion tracking for humanoid whole-body control. *arXiv preprint arXiv:2506.14770*, 2025.

[20] Z. Wang, J. Wang, S. Yao, Y. Zhang, Z. Ding, M. Yang, Y. Wang, H. Jiang, C. Ma, X. Shi, et al. General humanoid whole-body control via pretraining and fast adaptation. *arXiv preprint arXiv:2602.11929*, 2026.

[21] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn. Gradient surgery for multi-task learning. In *Proc. Advances in Neural Information Processing Systems*, 2020.

[22] B. Liu, X. Liu, X. Jin, P. Stone, and Q. Liu. Conflict-averse gradient descent for multi-task learning. In *Proc. Advances in Neural Information Processing Systems*, 2021.

[23] T. Schaul, D. Borsa, J. Modayil, and R. Pascanu. Ray interference: a source of plateaus in deep reinforcement learning. *arXiv preprint arXiv:1904.11455*, 2019.

[24] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.

[25] T. Willi, J. S. O. Ceron, J. N. Foerster, G. K. Dziugaite, and P. S. Castro. Mixture of experts in a mixture of RL settings. *Reinforcement Learning Journal*, 3:1072–1105, 2024.

[26] A. Hendawy, J. Peters, and C. D'Eramo. Multi-task reinforcement learning with mixture of orthogonal experts. In *Proc. International Conference on Learning Representations*, 2024.

[27] G. Cheng, L. Dong, W. Cai, and C. Sun. Multi-task reinforcement learning with attention-based mixture of experts. *IEEE Robotics and Automation Letters*, 8(6):3812–3819, 2023.

[28] R. Yang, H. Xu, Y. Wu, and X. Wang. Multi-task reinforcement learning with soft modularization. In *Proc. Advances in Neural Information Processing Systems*, 2020.

[29] R. Huang, S. Zhu, Y. Du, and H. Zhao. MoE-Loco: Mixture of experts for multitask locomotion. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2025.

[30] J. Han, W. Xie, J. Zheng, J. Shi, W. Zhang, T. Xiao, and C. Bai. KungfuBot2: Learning versatile motion skills for humanoid whole-body control. *arXiv preprint arXiv:2509.16638*, 2025.

[31] Y. Li, Y. Lin, J. Cui, T. Liu, W. Liang, Y. Zhu, and S. Huang. CLONE: Closed-loop whole-body humanoid teleoperation for long-horizon tasks. In *Proc. Conference on Robot Learning*, 2025.

[32] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *Proc. International Conference on Machine Learning*, 2015.

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[34] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proc. International Conference on Learning Representations*, 2017.

[35] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *Proc. International Conference on Learning Representations*, 2021.

[36] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[37] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. M. Dai, Q. V. Le, J. Laudon, et al. Mixture-of-Experts with expert choice routing. In *Proc. Advances in Neural Information Processing Systems*, 2022.

[38] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. Susano Pinto, D. Keysers, and N. Houlsby. Scaling vision with sparse mixture of experts. In *Proc. Advances in Neural Information Processing Systems*, 2021.

[39] J. Ren, Y. Li, Z. Ding, W. Pan, and H. Dong. Probabilistic mixture-of-experts for efficient deep reinforcement learning. *arXiv preprint arXiv:2104.09122*, 2021.

[40] R. Akrour, D. Tateo, and J. Peters. Continuous action reinforcement learning from a mixture of interpretable experts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44 (10):6795–6806, 2022.

[41] X. B. Peng, G. Berseth, and M. Van de Panne. Terrain-adaptive locomotion skills using deep reinforcement learning. *ACM Transactions on Graphics*, 35(4):1–12, 2016.

[42] J. Puigcerver, C. R. Ruiz, B. Mustafa, and N. Houlsby. From sparse to soft mixtures of experts. In *Proc. International Conference on Learning Representations*, 2024.

[43] B. Yang, G. Bender, Q. V. Le, and J. Ngiam. CondConv: Conditionally parameterized convolutions for efficient inference. In *Proc. Advances in Neural Information Processing Systems*, 2019.

[44] Z. Tian, C. Shen, and H. Chen. Conditional convolutions for instance segmentation. In *Proc. European Conference on Computer Vision*, 2020.

[45] M. Muqeeth, H. Liu, and C. Raffel. Soft merging of experts with adaptive routing. *Transactions on Machine Learning Research*, 2024.

[46] J. Obando-Ceron, G. Sokar, T. Willi, C. Lyle, J. Farebrother, J. Foerster, K. Dziugaite, D. Precup, and P. S. Castro. Mixtures of experts unlock parameter scaling for deep RL. In *Proc. International Conference on Machine Learning*, 2024.

[47] S. Jeon, M. Jung, S. Choi, B. Kim, and J. Hwangbo. Learning whole-body manipulation for quadrupedal robot. *IEEE Robotics and Automation Letters*, 9(1):699–706, 2024.

[48] Z. He, K. Lei, Y. Ze, K. Sreenath, Z. Li, and H. Xu. Learning visual quadrupedal loco-manipulation from demonstrations. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024.

[49] Z. Fu, X. Cheng, and D. Pathak. Deep whole-body control: Learning a unified policy for manipulation and locomotion. In *Proc. Conference on Robot Learning*, 2023.

[50] T. Portela, A. Cramariuc, M. Mittal, and M. Hutter. Whole-body end-effector pose tracking. In *Proc. IEEE International Conference on Robotics and Automation*, 2025.

[51] S. Lim, H. Lee, S. Lee, I. M. A. Nahrendra, and H. Myung. OpenHEART: Opening heterogeneous articulated objects with a legged manipulator. In *Proc. IEEE International Conference on Robotics and Automation*, 2026.

[52] S. Lim, H. Lee, S. Lee, and H. Myung. Learning robust whole-body control for legged manipulation via learned terrain estimation. In *Proc. International Conference on Control, Automation, and Systems*, 2024.

[53] A. E. Vijayan, A. Cramariuc, M. Risiglione, C. Gehring, and M. Hutter. Multi-critic learning for whole-body end-effector twist tracking. In *Proc. Conference on Robot Learning*, 2025.

[54] Z. Luo, J. Cao, A. Winkler, K. Kitani, and W. Xu. Perpetual humanoid control for real-time simulated avatars. In *Proc. IEEE/CVF International Conference on Computer Vision*, 2023.

[55] T. He, Z. Luo, X. He, W. Xiao, C. Zhang, W. Zhang, K. M. Kitani, C. Liu, and G. Shi. OmniH2O: Universal and dexterous human-to-humanoid whole-body teleoperation and learning. In *Proc. Conference on Robot Learning*, 2025.

[56] T. He, W. Xiao, T. Lin, Z. Luo, Z. Xu, Z. Jiang, J. Kautz, C. Liu, G. Shi, X. Wang, et al. HOVER: Versatile neural whole-body controller for humanoid robots. In *Proc. IEEE International Conference on Robotics and Automation*, 2025.

[57] J. Li, X. Cheng, T. Huang, S. Yang, R.-Z. Qiu, and X. Wang. AMO: Adaptive motion optimization for hyper-dexterous humanoid whole-body control. *arXiv preprint arXiv:2505.03738*, 2025.

[58] Y. Ze, S. Zhao, W. Wang, A. Kanazawa, R. Duan, P. Abbeel, G. Shi, J. Wu, and C. K. Liu. TWIST2: Scalable, portable, and holistic humanoid data collection system. *arXiv preprint arXiv:2511.02832*, 2025.

[59] J. P. Araujo, Y. Ze, P. Xu, J. Wu, and C. K. Liu. Retargeting matters: General motion retargeting for humanoid motion tracking. *arXiv preprint arXiv:2510.02252*, 2025.

[60] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, 1998.

[61] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proc. International Conference on Artificial Intelligence and Statistics*, 2011.

[62] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Munoz, X. Yao, R. Zurbrügg, N. Rudin, et al. Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025.

[63] Unitree Robotics. Unitree G1. https://www.unitree.com/g1. Accessed: 2026-05-22.

[64] N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proc. Conference on Robot Learning*, 2022.

[65] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine. Diversity is all you need: Learning skills without a reward function. In *Proc. International Conference on Learning Representations*, 2019.

[66] S. Park, O. Rybkin, and S. Levine. METRA: Scalable unsupervised RL with metric-aware abstraction. In *Proc. International Conference on Learning Representations*, 2024.

[67] D. Strouse, K. Baumli, D. Warde-Farley, V. Mnih, and S. S. Hansen. Learning more skills through optimistic exploration. In *Proc. International Conference on Learning Representations*, 2022.

[68] J. Hu, Z. Wang, P. Stone, and R. Martín-Martín. Disentangled unsupervised skill discovery for efficient hierarchical reinforcement learning. In *Proc. Advances in Neural Information Processing Systems*, 2024.

[69] R. Cathomen, M. Mittal, M. Vlastelica, and M. Hutter. Divide, discover, deploy: Factorized skill learning with symmetry and style priors. In *Proc. Conference on Robot Learning*, 2025.

[70] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *Proc. International Conference on Learning Representations*, 2015.

[71] D.-A. Clevert, T. Unterthiner, and S. Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *Proc. International Conference on Learning Representations*, 2016.

[72] S. Lee, S. Purushwalkam, M. Cogswell, D. Crandall, and D. Batra. Why M heads are better than one: Training a diverse ensemble of deep networks. *arXiv preprint arXiv:1511.06314*, 2015.

[73] C. D'Eramo, D. Tateo, A. Bonarini, M. Restelli, and J. Peters. Sharing knowledge in multi-task deep reinforcement learning. In *Proc. International Conference on Learning Representations*, 2020.

Supplementary Material

A Implicit Trust Region from clipping

Let $r_t = 1+\delta \quad (|\delta| \leq \epsilon)$, then the KL divergence between the new and old policies can be expressed as,

$$D(\pi^{\text{old}}||\pi^{\text{new}}) = \mathbb{E}\left[\log \frac{1}{r_t}\right] = \mathbb{E}\left[-\log(1+\delta)\right]. \tag{8}$$

Because $\log(1+\delta) \approx \delta - \delta^2/2$ for small δ, we have,

$$D(\pi^{\text{old}}||\pi^{\text{new}}) \approx \mathbb{E}\left[-\delta + \delta^2/2\right]. \tag{9}$$

Because $\mathbb{E}_{a\sim\pi^{\text{old}}(\cdot|s)}[\delta] = \int \pi_\Theta(a|s)\, da - 1 = 0$, the KL divergence can be approximated as follows:

$$D(\pi^{\text{old}}||\pi^{\text{new}}) \approx \mathbb{E}\left[\delta^2/2\right] = O(\delta^2) \leq O(\epsilon^2). \tag{10}$$

It means the clipping of PPO implicitly constructs a trust region by constraining the KL divergence to $O(\epsilon^2)$.

B Bias Analysis of MAPO

Recall that the dense MoE policy can be modeled as $\pi_\Theta(a|s) = \mathcal{N}(m_\Theta, \sigma^2 I)$, where $m_\Theta = \sum_e w_e \mu_e$ and $\Theta = \{\psi, \theta_{1:N}\}$. The mixing weights $\mathbf{w} = [w_1, w_2, \ldots, w_N]$ are obtained by applying a softmax to the output of the router. The e-th expert outputs the deterministic mean $\mu_e = \mu_e(s;\theta_e)$. For simplicity, we assume that all experts share the same variance σ^2.

B.1 Score Function Decomposition

Consider the score function decomposition for the policy gradient algorithm. Define $h(s,a)$ as,

$$\nabla_m \log \pi(a|s) = \frac{a-m}{\sigma^2} \doteq h(s,a). \tag{11}$$

Then,

$$\nabla_\Theta \log \pi_\Theta(a|s) = \nabla_m \log \pi(a|s)\, \nabla_\Theta m_\Theta \tag{12}$$

$$= h(s,a) \nabla_\Theta m_\Theta. \tag{13}$$

The ∇_Θ consists of gradients for the parameters of the router and the experts, which can be expressed as,

$$\nabla_\psi m_\Theta = \sum_e \frac{\partial w_e}{\partial \psi} \mu_e, \text{ and } \nabla_{\theta_k} m_\Theta = w_k \frac{\partial \mu_k}{\partial \theta_k}. \tag{14}$$

Therefore,

$$\nabla_\psi \log \pi_\Theta(a|s) = h \sum_e \frac{\partial w_e}{\partial \psi} \mu_e, \text{ and } \nabla_{\theta_k} \log \pi_\Theta(a|s) = h w_k \frac{\partial \mu_k}{\partial \theta_k}. \tag{15}$$

B.2 PPO Gradient

Because the gradient of PPO when it is clipped is zero, we only consider the case when it is not clipped. In this case, the PPO gradient is identical to the gradient of true importance sampling.

Recall,

$$L^{\text{PPO}} = \hat{\mathbb{E}}\left[\min\left(r(s,a)\hat{A}, \text{clip}(r(s,a), 1-\epsilon, 1+\epsilon)\hat{A}\right)\right] \tag{16}$$

where $r(s,a) = \pi_\Theta(a|s)/\pi^{\text{old}}(a|s)$.

Notably,

$$\nabla_\Theta L^{\text{PPO}} = \hat{\mathbb{E}}\left[\nabla_\Theta r(s,a)\hat{A}\right] = \hat{\mathbb{E}}\left[r(s,a)h(s,a)\nabla_\Theta m(s;\Theta)\hat{A}\right] \tag{17}$$

B.3 MAPO Gradient

Let, $m_1(s;\psi) = \sum_e w_e(s,\psi)\mu_e^{\text{old}}(s)$ and $m_2(s;\theta_{1:N}) = \sum_e w_e^{\text{old}}(s)\mu_e(s;\theta_{1:N})$. Then, we can define the following quantities,

$$\pi_1 = \mathcal{N}(m_1, \sigma^2 I), \quad r_1 = \pi_1/\pi^{\text{old}}, \quad h_1 = (a - m_1)/\sigma^2 \tag{18}$$

$$\pi_2 = \mathcal{N}(m_2, \sigma^2 I), \quad r_2 = \pi_2/\pi^{\text{old}}, \quad h_2 = (a - m_2)/\sigma^2. \tag{19}$$

Similar to the PPO case, we only consider the case when the MAPO gradient is not clipped. Then, the MAPO loss function can be expressed as,

$$L^{\text{MAPO}} = \hat{\mathbb{E}}\left[r_1(s,a)\hat{A}\right] + \hat{\mathbb{E}}\left[r_2(s,a)\hat{A}\right]. \tag{20}$$

Then the gradient of MAPO with respect to each parameter can be expressed as,

$$\nabla_\psi L^{\text{MAPO}} = \hat{\mathbb{E}}\left[r_1 h_1(s,a)\nabla_\psi m_1(s;\psi)\hat{A}\right] \tag{21}$$

$$\nabla_{\theta_k} L^{\text{MAPO}} = \hat{\mathbb{E}}\left[r_2 h_2(s,a)\nabla_{\theta_k} m_2(s;\theta_{1:N})\hat{A}\right], \tag{22}$$

where $\nabla_\psi m_1(s;\psi) = \sum_e \frac{\partial w_e}{\partial \psi}\mu_e^{\text{old}}$ and $\nabla_{\theta_k} m_2(s;\theta_{1:N}) = w_k^{\text{old}}\frac{\partial \mu_k}{\partial \theta_k}$.

B.4 Effect of Clipping

Because the clipping implicitly constrains the KL divergence induced by each of the two intermediate ratios r_1, r_2 to $O(\epsilon^2)$, it yields the following inequalities:

$$\frac{||m_1 - m_{\text{old}}||}{\sigma} = \frac{1}{\sigma}||\sum(w_e - w_e^{\text{old}})\mu_e^{\text{old}}|| \leq \epsilon \tag{23}$$

$$\frac{||m_2 - m_{\text{old}}||}{\sigma} = \frac{1}{\sigma}||\sum(\mu_e - \mu_e^{\text{old}})w_e^{\text{old}}|| \leq \epsilon. \tag{24}$$

B.5 Bias Analysis

There are two sources of bias in the MAPO gradient. The first is the bias in the mean gradient term, ∇m. The second is the bias in the importance sampling ratio, r, and score factor h. As we will show, the second source is of the same order as the surrogate approximation error of PPO and does not change the qualitative picture; we focus first on the bias in the mean gradient term.

Mean Gradient Bias in Expert Parameters: We have,

$$\nabla_{\theta_k} m_\Theta = w_k \frac{\partial \mu_k}{\partial \theta_k} \quad \text{(for PPO)} \tag{25}$$

$$\nabla_{\theta_k} m_2 = w_k^{\text{old}} \frac{\partial \mu_k}{\partial \theta_k} \quad \text{(for MAPO)}. \tag{26}$$

Therefore, the MAPO gradient on each expert block is the true importance sampling gradient on that block scaled by a per-expert positive factor w_k^{old}/w_k. As these scalars differ across experts, the aggregate gradient direction in $\theta_{1:N}$-space is not preserved in general; however, each per-expert block vanishes exactly when its true counterpart does, therefore the stationary points of MAPO coincide with those of the true importance sampling gradient.

Mean Gradient Bias in Router Parameters: The bias in the mean gradient term for the router parameters can be expressed as,

$$b_\psi^{\text{mean}} = \sum_e \frac{\partial w_e}{\partial \psi}(\mu_e - \mu_e^{\text{old}}). \tag{27}$$

Notably, unlike the expert parameters, the MAPO gradient with respect to the router parameters can have a directional bias. The following cancellation shows that this bias depends only on the *relative* deviations between experts, not on their absolute magnitudes.

Because the sum of the mixing weights is 1, we have,

$$\sum_e \frac{\partial w_e}{\partial \psi} = \frac{\partial}{\partial \psi} \sum_e w_e = 0. \tag{28}$$

Then, for any constant $\bar{\xi}$ independent of the expert index e, we have

$$\sum_e \frac{\partial w_e}{\partial \psi}(\mu_e - \mu_e^{\text{old}}) = \sum_e \frac{\partial w_e}{\partial \psi}\left[(\mu_e - \mu_e^{\text{old}}) - \bar{\xi}\right], \qquad \text{because } \sum_e \frac{\partial w_e}{\partial \psi}\bar{\xi} = \bar{\xi}\sum_e \frac{\partial w_e}{\partial \psi} = 0. \tag{29}$$

We define

$$\xi_e = (\mu_e - \mu_e^{\text{old}}), \text{ and } \bar{\xi} = \frac{1}{N}\sum_e \xi_e = \frac{1}{N}\sum_e (\mu_e - \mu_e^{\text{old}}), \tag{30}$$

then, we have

$$b_\psi^{\text{mean}} = \sum_e \frac{\partial w_e}{\partial \psi}(\xi_e - \bar{\xi}). \tag{31}$$

Therefore, the bias depends on the relative difference between each expert, rather than the absolute value of the difference. In particular, when all the experts are updated coherently, $\xi_e - \bar{\xi} \approx 0$ and the bias vanishes.

Bound on the Rotation Angle on the Router Gradient: We now show that the directional discrepancy between the MAPO router gradient $\hat{g} := \nabla_\psi m_1$ and the true mean gradient $g := \nabla_\psi m_\Theta$ is also controlled by the inter-expert variance. Within a single update step, the trust region in ψ keeps the router parameters close to their old values, and the router Jacobian is approximately unchanged: $\partial w_e/\partial\psi \approx \partial w_e^{\text{old}}/\partial\psi$. Under this approximation, the gradient discrepancy reduces to the mean-gradient bias,

$$g - \hat{g} \;\approx\; b_\psi^{\text{mean}}. \tag{32}$$

Decomposing b_ψ^{mean} into components parallel and perpendicular to $\hat{g}$, the angle θ between g and $\hat{g}$ satisfies

$$\sin\theta \;\lesssim\; \frac{\|b_\psi^{\text{mean}}\|}{\|\hat{g}\|}, \tag{33}$$

where the $\lesssim$ absorbs only the first-order Jacobian approximation above.

Applying the Cauchy–Schwarz inequality to (31),

$$\|b_\psi^{\text{mean}}\| \;\le\; \sqrt{\sum_e \left\|\tfrac{\partial w_e}{\partial \psi}\right\|^2} \cdot \sqrt{\sum_e \|\xi_e - \bar{\xi}\|^2}. \tag{34}$$

Because $\hat{g} = \sum_e (\partial w_e^{\text{old}}/\partial\psi)\,\mu_e^{\text{old}}$ shares the same router Jacobian as the first factor above, that factor cancels to leading order in the ratio $\|b_\psi^{\text{mean}}\|/\|\hat{g}\|$. The rotation angle is therefore governed primarily by the inter-expert variance $\sum_e \|\xi_e - \bar{\xi}\|^2$, not by the absolute expert deviations or the router sensitivity. Whether this variance remains small in practice depends on the training dynamics and is not guaranteed by the trust-region constraint $\|\sum_e w_e^{\text{old}}\xi_e\| \le \epsilon\sigma$ from Equation 24 alone. Figure B.1 shows the changes in the router-gradient angle during training. The router-gradient angle remains below 90° throughout training.

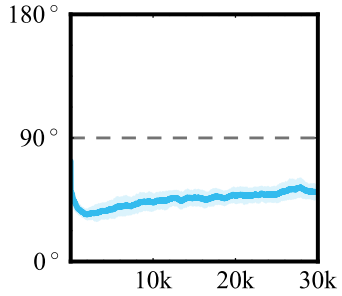


Figure B.1: Router-gradient angle.

Table C.1: Reward functions for teacher training. Superscripts p and g denote proprioception and goal states, respectively. $\mathbf{o}$, $\mathbf{p}$, $\mathbf{v}$, $\boldsymbol{\omega}$, $\mathbf{a}$, $\mathbf{q}$, and $\mathbf{F}$ are orientation, position, linear velocity, angular velocity, policy action, joint position, and contact force. Subscripts specify the reference point: *root* for the pelvis link, *body* for tracked body links in $\mathcal{B}$, *rel-body* for body link quantities expressed relative to the root frame, and *last* for the previous timestep. $\mathbb{I}[\cdot]$ is the indicator function.

Reward term	**Equation**	**Weight**
Root orientation	$\exp\left(-\|\|\mathbf{o}^{p}_{\text{root}} - \mathbf{o}^{g}_{\text{root}}\|\|_2^2/0.4^2\right)$	1.0
Relative body-link position	$\exp\left(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\|\mathbf{p}^{p}_{\text{rel-body},b} - \mathbf{p}^{g}_{\text{rel-body},b}\|\|_2^2/0.05^2\right)$	20.0
Relative body-link orientation	$\exp\left(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\|\mathbf{o}^{p}_{\text{rel-body},b} - \mathbf{o}^{g}_{\text{rel-body},b}\|\|_2^2/0.4^2\right)$	1.0
Body-link linear velocity	$\exp\left(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\|\mathbf{v}^{p}_{\text{body},b} - \mathbf{v}^{g}_{\text{body},b}\|\|_2^2/1.0\right)$	1.0
Body-link angular velocity	$\exp\left(-\frac{1}{\|\mathcal{B}\|}\sum_{b\in\mathcal{B}}\|\|\boldsymbol{\omega}^{p}_{\text{body},b} - \boldsymbol{\omega}^{g}_{\text{body},b}\|\|_2^2/3.14^2\right)$	1.0
Action rate	$\|\|\mathbf{a} - \mathbf{a}_{\text{last}}\|\|_2^2$	-0.2
Joint limit	$\sum_j \mathbb{I}\left[\mathbf{q}_j \notin [\mathbf{q}_j^{\min}, \mathbf{q}_j^{\max}]\right]$	-10.0
Undesired contact	$\sum_{c\notin\{\text{ankles, wrists}\}} \mathbb{I}\left[\|\|\mathbf{F}_c\|\| > 1.0N\right]$	-0.1

Difference in Importance Sampling Ratio and Score Factor: Because of the clipping, the deviations in the importance sampling ratio and score factor are of order $O(\epsilon)$,

$$|r_i - r| \approx O(\epsilon), \text{ and } |h_i - h| = |m_i - m|/\sigma^2 \approx O(\epsilon) \quad (i = 1, 2). \tag{35}$$

Decomposing the gradient difference as

$$r_i h_i \nabla m_i - rh\nabla m = rh\left(\nabla m_i - \nabla m\right) + \left(r_i h_i - rh\right)\nabla m_i,$$

the first term is the mean-gradient bias analyzed above, while the second contributes an additional $O(\epsilon)$ correction because $r_i h_i - rh = O(\epsilon)$. Importantly, within the trust region $r \approx 1$, therefore the leading rh pre-factor is itself close to the score h, and the residual error from the (r_i, h_i) deviation is on the same order as PPO's surrogate approximation error and does not alter the qualitative conclusions of the mean-gradient analysis.

C Training Details

C.1 Reward Function

The reward function for the teacher training is almost similar to the prior works [15, 17]. The specific reward functions are listed in Table C.1. We find that the sharper relative body link position reward encourages the robot to fully complete the reference motion, particularly for deep squatting and bowing. Without it, the robot tends to follow the reference motion only partially, stopping halfway through such motions, as shown in Figure C.1 (b). Notably, **MAPO** also achieves higher reward than **PPO with MoE** under the BeyondMimic reward setting (Figure C.1 (c)).

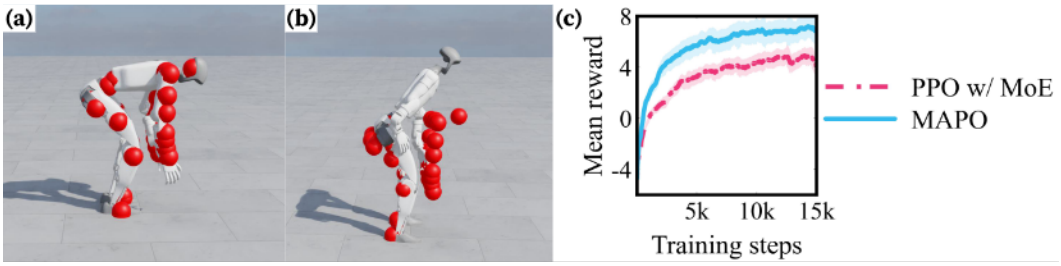


Figure C.1: Comparison of reward settings and training performance. (a) The reward setting used in this work. (b) The reward setting of BeyondMimic [15]. The reward settings differ mainly in the scale and sharpness of the relative body-link position reward. (c) Reward learning curves of MAPO and PPO with MoE under the BeyondMimic reward setting.

Table C.2: Teacher policy training hyperparameters.

Hyperparameter	Value
Num parallel environments	4096
Num steps per environment	24
Num mini-batches	4
Learning epochs	5
Discount γ	0.99
GAE λ	0.95
Clip parameter ϵ	0.2
Entropy coefficient	0.01
Learning rate	Adaptive
Max grad norm	1.0
Init noise std	1.0
Shared actor encoder hidden layers	[512, 256]
Shared encoder final feature dimension	256
Router hidden layers	[256, 128]
Expert hidden layers	[128, 128]
Value function hidden layers	[512, 256, 128]

C.2 Training Hyperparameters

The training hyperparameters for the teacher policy are listed in Table C.2. The Adam [70] optimizer is used for optimization, and exponential linear units (ELUs) [71] are used as the activation function in all networks. Additionally, the teacher uses a shared encoder that extracts features from the observation, which are fed into both the router and experts [72, 73]. Notably, for the alternating optimization, we detach the gradients from the router to the shared encoder. For the student policy distillation, a fixed learning rate of 2×10^{-4} is used, and the student policy network adopts hidden layers of sizes $[512, 256, 128]$.

C.3 Domain Randomization

To improve the robustness of the trained policy, we apply domain randomization during training. Details of the randomized parameters are listed in Table C.3.

D Additional Experimental Results

Per-Expert Activation: Figure D.1 shows per-expert activation across different motion types. The router assigns higher mixing weights to different experts depending on the motion type, indicating expert specialization. Expert 2 (E2) is primarily activated during walking and upper-body motions

Table C.3: Domain randomization parameters. $\mathcal{U}(a, b)$ denotes a uniform distribution over the interval $[a, b]$.

Domain Randomization Parameter	Value
Joint P gain scale	$\mathcal{U}(0.75, 1.25) \times$ default
Joint D gain scale	$\mathcal{U}(0.75, 1.25) \times$ default
Root mass	$\mathcal{U}(0.75, 1.25)$ + default kg
Root center of mass offset (x, y, z)	$\mathcal{U}([-0.1, 0.1], [-0.1, 0.1], [-0.1, 0.1])$ m
Control delay	$\mathcal{U}(0, 60)$ ms
Push robot magnitude	$\mathcal{U}(-1.0, 1.0)$ m/s
Push robot interval	5 s
Reference motion offset (x, y, z)	$\mathcal{U}([-0.02, 0.02], [-0.02, 0.02], [-0.1, 0.1])$ m

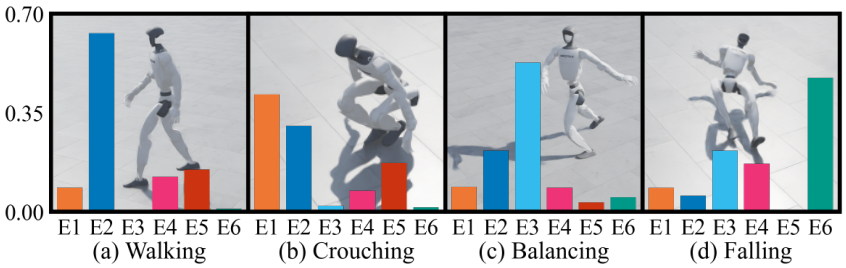


Figure D.1: Per-expert activation according to the motion type.

while standing (Figure D.1 (a)). E4 and E5 support E2 during different phases of walking. Although E1 appears redundant in Figure 4 (c), E1 is activated for motions that require a squatting posture (Figure D.1 (b)). E3 and E6 are activated during balance recovery and unavoidable falls, respectively (Figure D.1 (c, d)). Because we do not manually curate the dataset, some reference motions may be infeasible for the robot. Concentrating activation during unavoidable falls in E6 may reduce interference from infeasible motions during the training of the remaining experts.

Effect of Divergence Misestimation: We measure divergence misestimation during training with vanilla policy gradient because clipping can obscure the magnitude of misestimation. Figure D.2 shows the mean and percentiles of overestimation and underestimation magnitudes. We also evaluate the fractions of updates that would be clipped if the PPO or MAPO criteria were applied during training with vanilla policy gradient (Figure D.3 (a)). The MAPO router criterion would clip only about 5% of updates. In contrast, if the PPO criterion were applied to the same updates, approximately 50% of updates would be clipped. The higher clipping fraction for router updates is consistent with the weaker expert specialization of **PPO with MoE** (Figure 4 (b, c)).

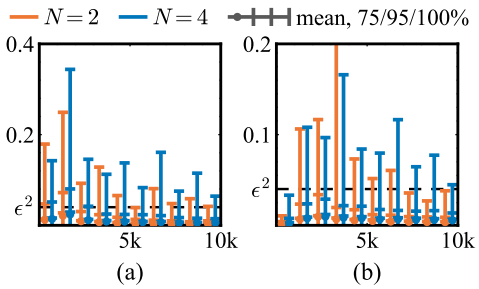


Figure D.2: Mean and percentiles of divergence misestimation magnitudes. (a) Overestimation. (b) Underestimation.

A similar pattern appears during **MAPO** training (Figure D.3 (c, d)). The MAPO router criterion clips only about 2% of updates, compared with approximately 24% if the PPO criterion were applied. Notably, the PPO criterion yields similar clipping fractions during **MAPO** training and during training of **PPO with MoE** (Figure D.3 (d)). The results suggest that excessive clipping of router updates is prevented during **MAPO** training.

Seed Variance: The results in Table 1 and Figure 4 are based on a single training seed. We further evaluate **MAPO** and **PPO with MoE** across three training seeds.

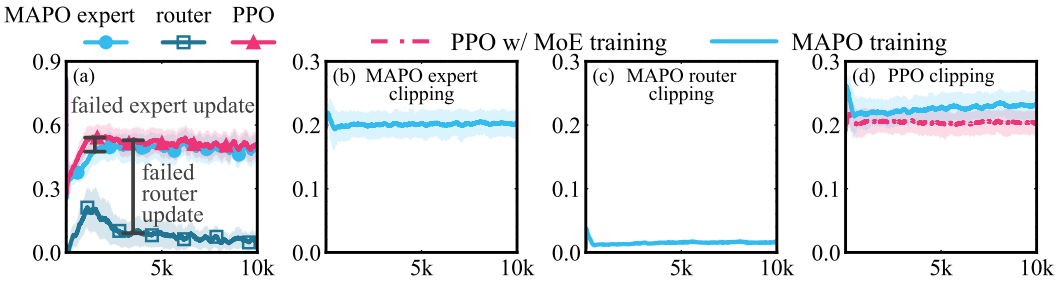


Figure D.3: (a) Fractions of updates that would be clipped under the MAPO and PPO criteria during training with vanilla policy gradient. (b, c) Clipped update fractions under the MAPO expert and router criteria, respectively, during **MAPO** training. (d) Clipping fractions under the PPO criterion during **MAPO** training (hypothetical clipping) and **PPO with MoE** training (actual clipping).

The learning curves are shown in Figure D.4. Across the three seeds, **MAPO** achieves a success rate of $(87.03 \pm 0.64)\%$ and a global body position error $E_{\text{g-pos}}$ of 147.9 ± 1.5 mm. The corresponding results for **PPO with MoE** are $(82.89 \pm 0.43)\%$ and 154.1 ± 1.7 mm. Success rates and tracking errors are reported as mean $\pm$ standard deviation across the three training seeds.

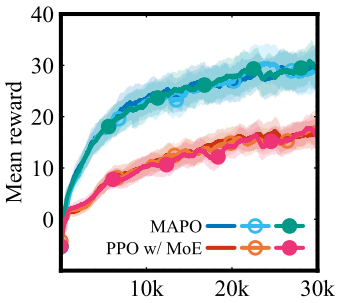


Figure D.4: Learning curves of **MAPO** and **PPO with MoE** across three training seeds.

Additionally, we examine whether the difference in tracking success between **MAPO** and **PPO with MoE** is consistent across training seeds. For each seed, we count motions on which **MAPO** succeeds and **PPO with MoE** fails in all five evaluation rollouts. The counts are 182, 155, and 191 for the three training seeds, respectively. Conversely, **PPO with MoE** consistently succeeds while **MAPO** consistently fails on only 1, 1, and 0 motions, respectively.